

法律文书标注平台 类图与 ER 图（实现版）

Phase 3 详细设计 · 第 44 组

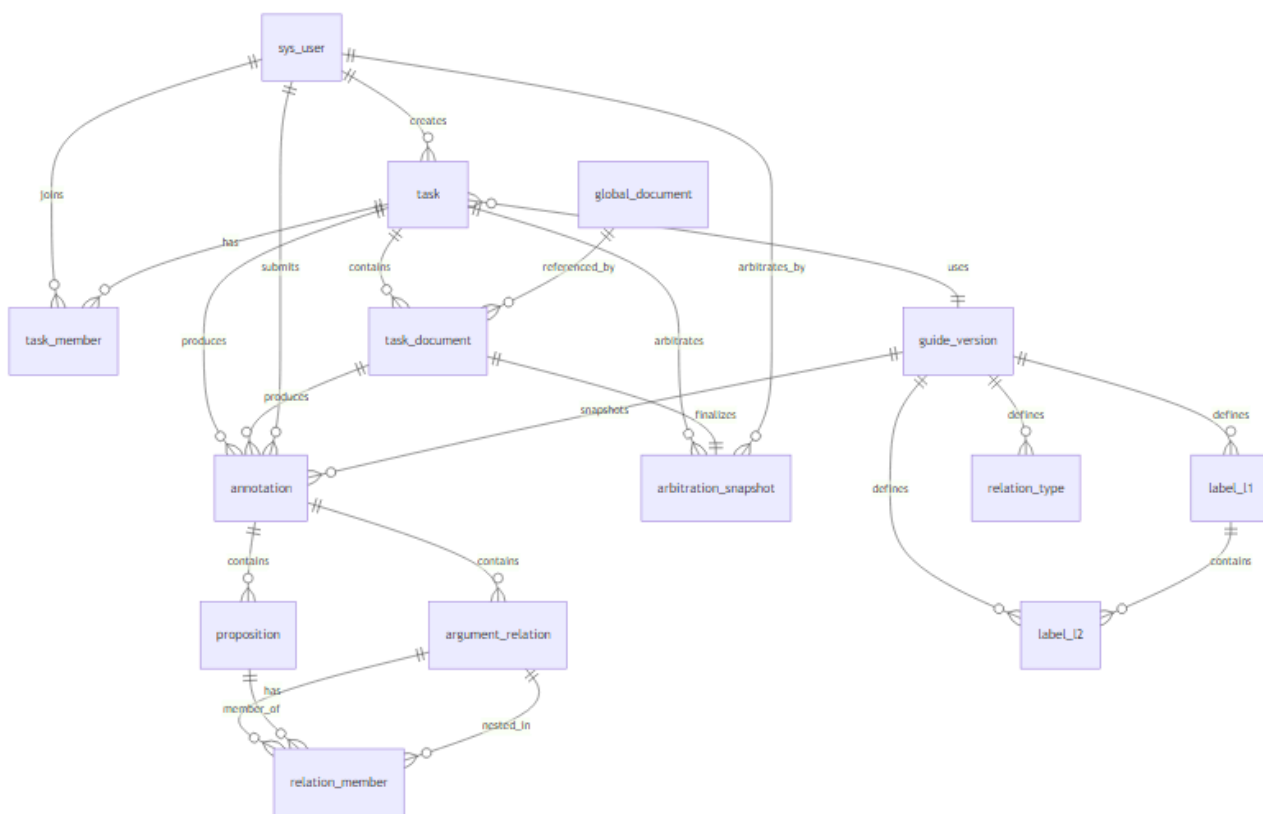
2026 年 6 月 16 日

本文档基于项目当前真实代码与数据库结构编写，对应 `deploy/init.sql` 与 `backend/src/main/java/edu/nju/jap/` 下的实现。

持久化实体（共 14 张表）：sys_user、guide_version、label_l1、label_l2、relation_type、global_document、task、task_member、task_document、annotation、proposition、argument_relation、relation_member、arbitration_snapshot。

一、ER 图

1.1 实体关系总览



ER 图

Mermaid 源文件：P3-ER图.mmd

1.2 各实体间关系

2	sys_user	task_member	参与	1:N	task_member.user_id → sys_user.id	标注员 / 裁定者
3	task	task_member	包含成员	1:N	task_member.task_id → task.id	任务级分配，非按文书分配
4	guide_version	label_l1	定义	1:N	label_l1.guide_version_id → guide_version.id	一级标签
5	guide_version	label_l2	定义	1:N	label_l2.guide_version_id → guide_version.id	二级标签（仅 GM）
6	label_l1	label_l2	包含	1:N	label_l2.parent_l1_id → label_l1.id	GM 下挂二级标签
7	guide_version	relation_type	定义	1:N	relation_type.guide_version_id → guide_version.id	关系类型
8	guide_version	task	被引用	1:N	task.guide_version_id → guide_version.id	任务绑定指南版本
9	task	task_document	包含	1:N	task_document.task_id → task.id	任务内文书
10	global_document	task_document	被引用	1:N	task_document.global_doc_id → global_document.id	来源为 GLOBAL / RECREATE 时
11	task	annotation	产生	1:N	annotation.task_id → task.id	标注 / 裁定记录
12	task_document	annotation	产生	1:N	annotation.document_id → task_document.id	按文书维度存储
13	sys_user	annotation	提交	1:N	annotation.user_id → sys_user.id	标注员或裁定者
14	guide_version	annotation	快照	N:1	annotation.guide_version_id → guide_version.id	提交时保存 guide_snapshot
15	annotation	proposition	包含	1:N	proposition.annotation_id → annotation.id	规范化存储，非 JSON 快照

16	annotation	argument_relation	包含	1:N	argument_relation.annotation_id → annotation.id	关系表
17	argument_relation	relation_member	包含成员	1:N	relation_member.relation_id → argument_relation.id	关系成员
18	proposition	relation_member	作为成员	1:N	relation_member.proposition_id → proposition.id	member_type='1'
19	argument_relation	relation_member	嵌套为成员	1:N	relation_member.child_relation_id → argument_relation.id	member_type='1' 支持关系嵌套
20	task	arbitration_snapshot	裁定	1:N	arbitration_snapshot.task_id → task.id	裁定元数据
21	task_document	arbitration_snapshot	最终裁定	1:1	arbitration_snapshot.task_document_id → task_document.id	每文书一条裁定记录
22	sys_user	arbitration_snapshot	执行	1:N	arbitration_snapshot.arbitrator_id → sys_user.id	裁定者

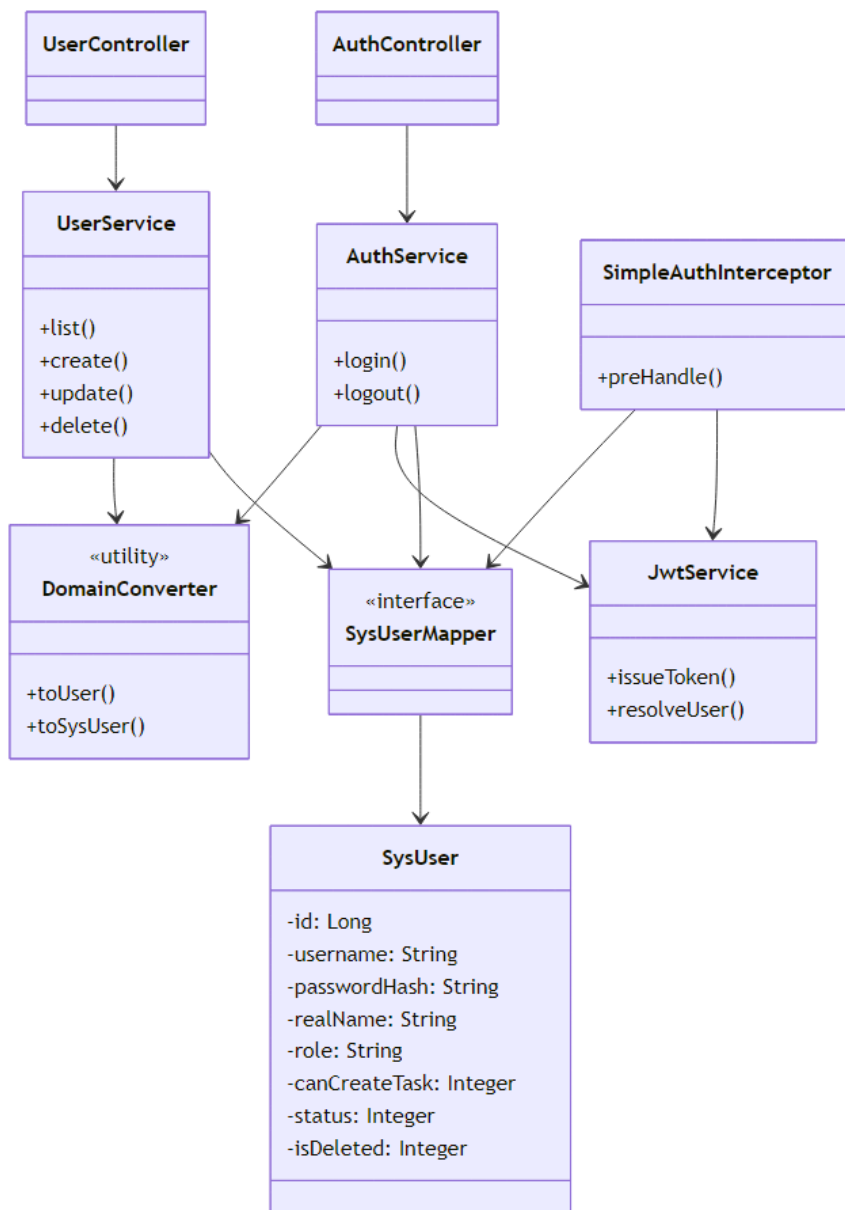
1.3 与初版设计的主要差异

初版设计（文档阶段）	当前实现	变更原因
sys_role 独立角色表 + role_id 外键	sys_user.role 字段（admin/creator/user）	角色种类少，简化权限模型
task_assignment（含 documentId、assignedBy）	task_member（任务级，无文书维度）	标注员对任务内全部文书负责
annotation 存 propositionData/relationData JSON	proposition / argument_relation / relation_member 规范化表	便于查询、比对、裁定差异分析

arbitration 独立表存 JSON 快照	annotation (record_type=ARBITRATION) + arbitration_snapshot 元数据	复用同一套持久化 逻辑
export_log / export_file	无（前端浏览器端 ZIP 导出）	降低后端复杂度
label_config 配置历史表	annotation.guide_snapshot JSON	提交时快照即可满 足追溯需求

二、类图总览

3.1 用户与认证

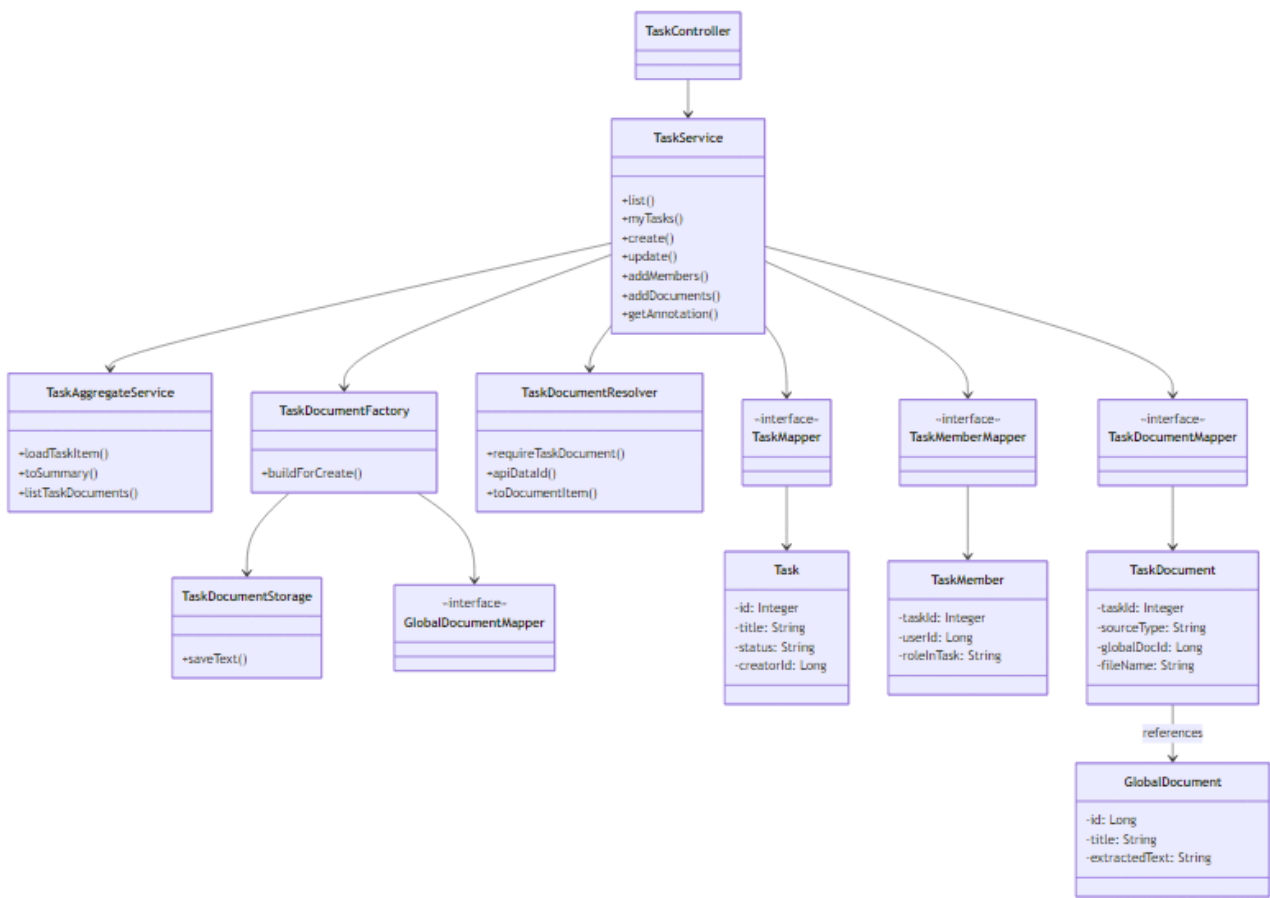


用户认证类图

Mermaid 源文件：P3-用户认证类图.mmd

角色不单独建表，通过 `sys_user.role` 与 `can_create_task` 控制权限；JWT 由 `SimpleAuthInterceptor` 统一校验。

3.2 任务与文书



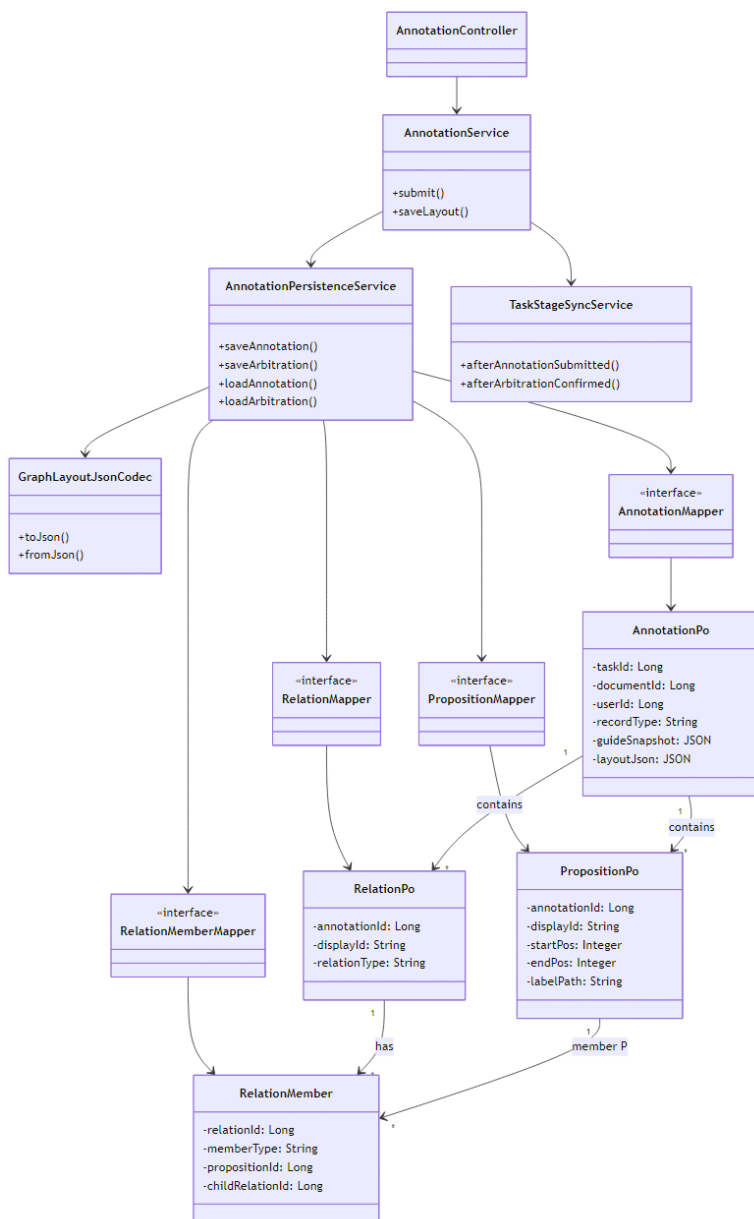
任务文书类图

Mermaid 源文件：P3-任务文书类图.mmd

TaskDocumentFactory（工厂模式）：根据 `sourceType` 创建不同来源的文书记录。

sourceType	行为
GLOBAL	引用 global_document，不复制文本
UPLOAD	上传文本，写入 TaskDocumentStorage
RECREATE	基于 global_document 重新编辑后保存

3.3 标注核心

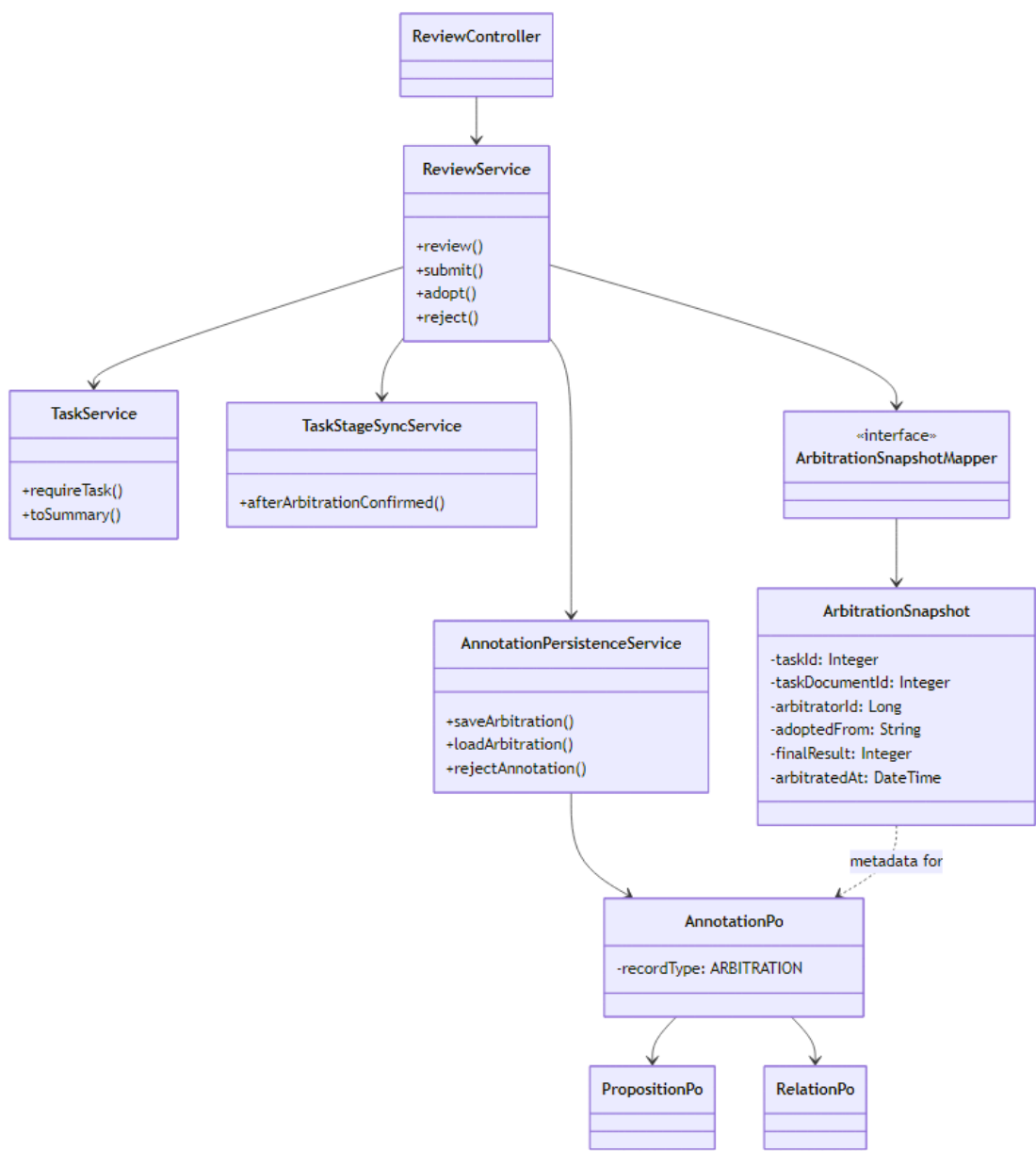


标注核心类图

Mermaid 源文件：P3-标注核心类图.mmd

命题与关系以规范化表存储，通过 `annotation_id` 关联；`record_type` 区分标注员提交与裁定者最终结果。

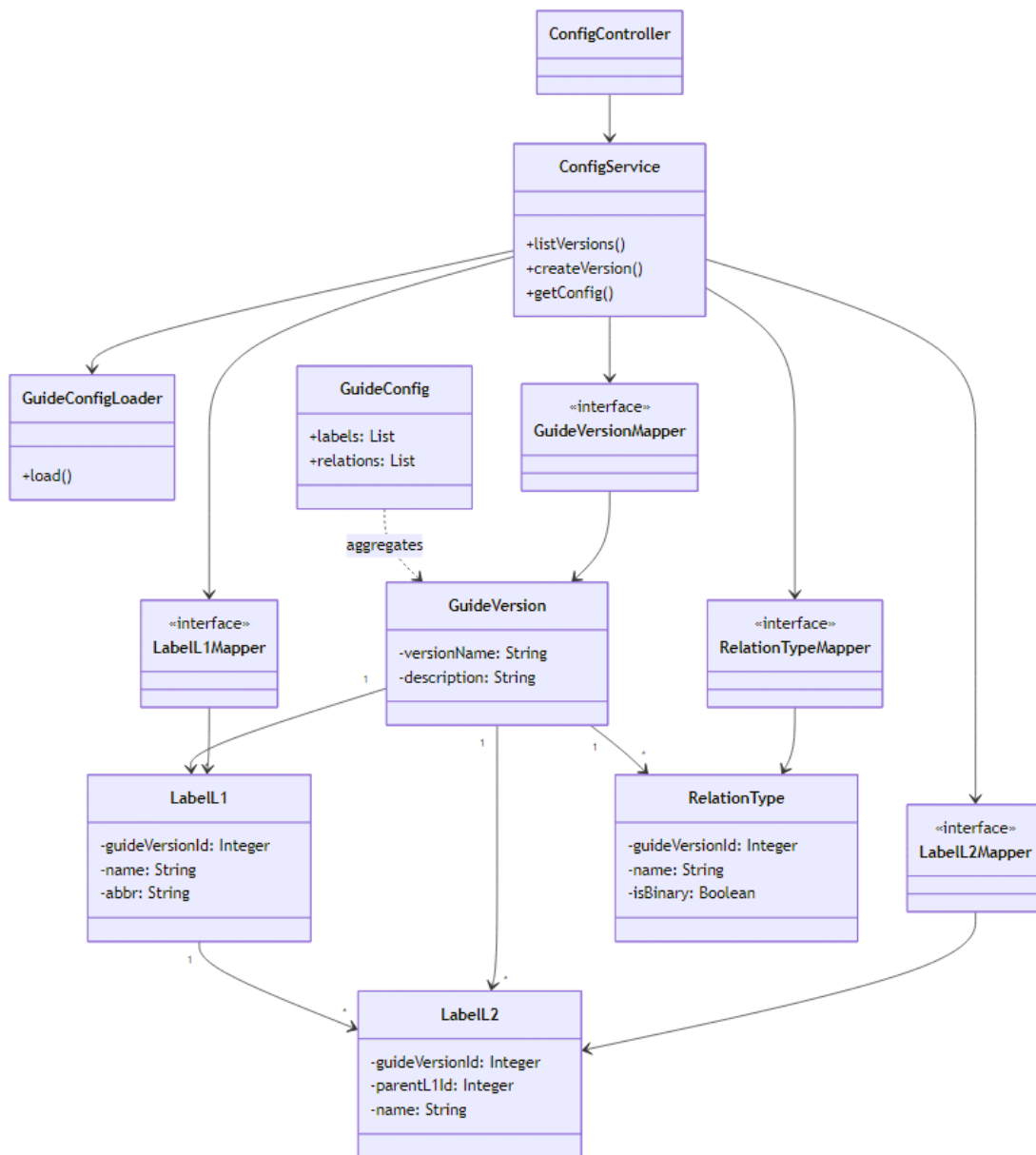
3.4 裁定



裁定类图

Mermaid 源文件：P3-裁定类图.mmd

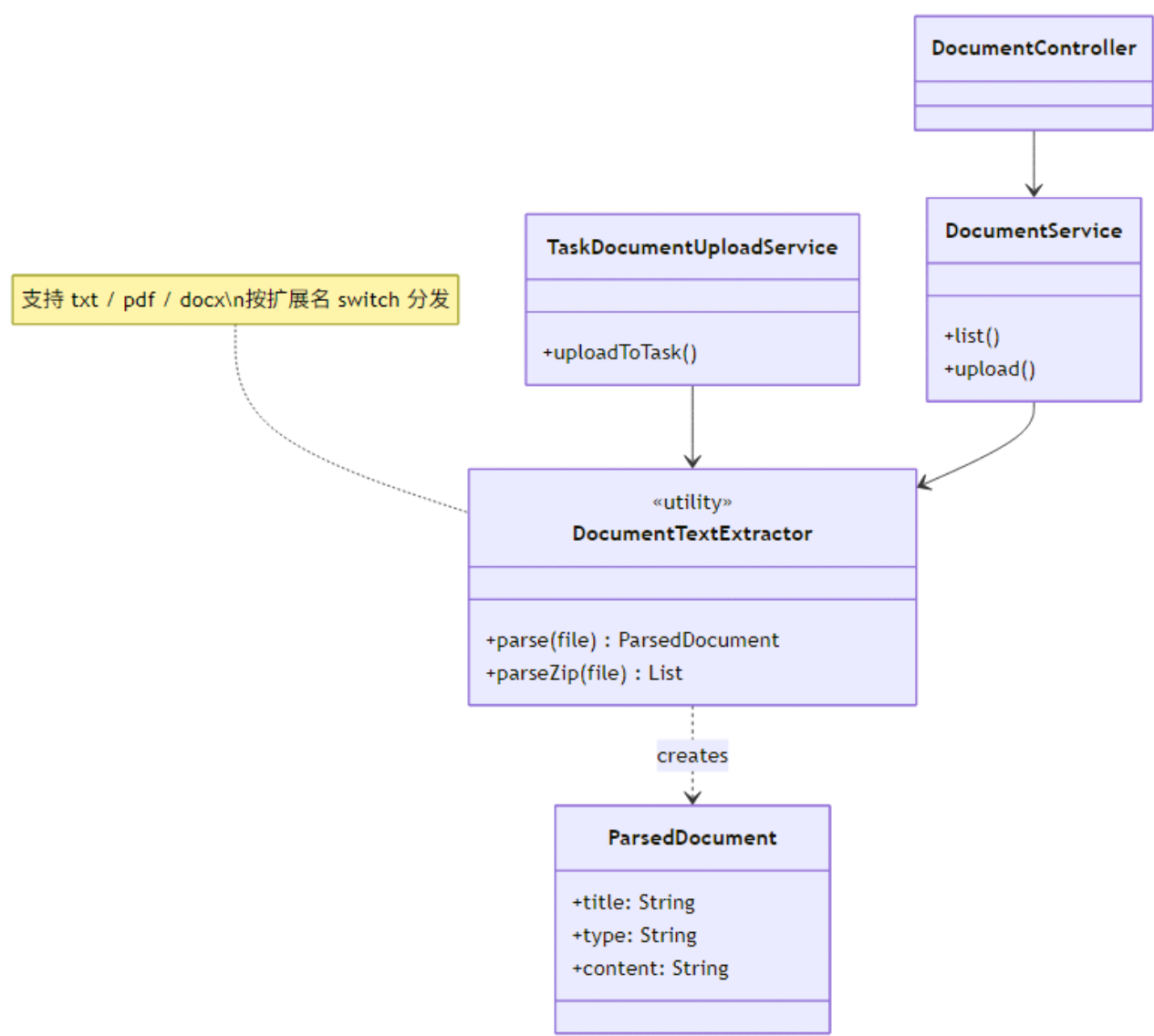
3.5 配置中心



配置中心类图

Mermaid 源文件：P3-配置中心类图.mmd

3.6 文书解析（工具类）



文书解析类图

Mermaid 源文件：P3-文书解析类图.mmd

四、ER 实体与代码类映射

序号	数据库表	PO 类	领域/DTO 类	说明
1	sys_user	SysUser	User, UserVO	用户；role 内嵌，无独立 Role 表

8	task_member	TaskMember	（嵌入 TaskItem）	任务成员
9	task_document	TaskDocument	DocumentItem	任务文书
10	annotation	AnnotationPo	AnnotationResult	标注/裁定记录
11	proposition	PropositionPo	Proposition	命题
12	argument_relation	RelationPo	Relation	论证关系
13	relation_member	RelationMember	（嵌入 Relation）	关系成员
14	arbitration_snapshot	ArbitrationSnapshot	ArbitrationResult	裁定元数据

五、设计模式应用（实际落地）

5.1 工厂模式 — TaskDocumentFactory

- 使用位置： `TaskDocumentFactory.buildForCreate()`
- 原因：任务文书有三种来源（GLOBAL / UPLOAD / RECREATE），创建逻辑差异大，集中在一处便于维护。
- 不用会怎样： `TaskService.create()` 内会出现大量 if-else，新增来源类型需改动核心服务。

5.2 分层架构 + 转换器 — DomainConverter

- 使用位置： `DomainConverter` 负责 PO ↔ Entity/DTO 转换
- 原因：持久化对象与 API 响应对象分离，Mapper 层不污染业务语义。
- 不用会怎样：PO 直接暴露给 Controller，字段命名与表结构强耦合，前端接口难以稳定。

5.3 阶段同步服务 — TaskStageSyncService

- 使用位置：标注提交、裁定确认后自动推进 task / task_document 状态
- 原因：状态流转规则（标注中 → 待裁定 → 可导出）集中管理，避免散落在多个 Service。
- 说明：未采用完整状态模式类层次，以实用服务类实现，降低代码量。

5.4 持久化聚合 — AnnotationPersistenceService

- 使用位置：统一处理 annotation + proposition + argument_relation + relation_member 的读写

- **原因：**四张表在同一事务内联动，聚合服务保证一致性。
- **不用会怎样：**AnnotationService 与 ReviewService 重复编写相同的级联保存逻辑。

六、类之间的关系汇总

6.1 持久化实体间关系（与 ER 图一致）

见 § 1.2 关系表。

6.2 服务层依赖关系

源服务	目标	说明
TaskService	TaskAggregateService, TaskDocumentFactory, AnnotationPersistenceService	任务 CRUD 与标注加载
AnnotationService	AnnotationPersistenceService, TaskStageSyncService	标注提交
ReviewService	AnnotationPersistenceService, TaskService, TaskStageSyncService	裁定流程
AuthService	JwtService, SysUserMapper, DomainConverter	登录认证
ConfigService	GuideConfigLoader, 各 Config Mapper	指南配置

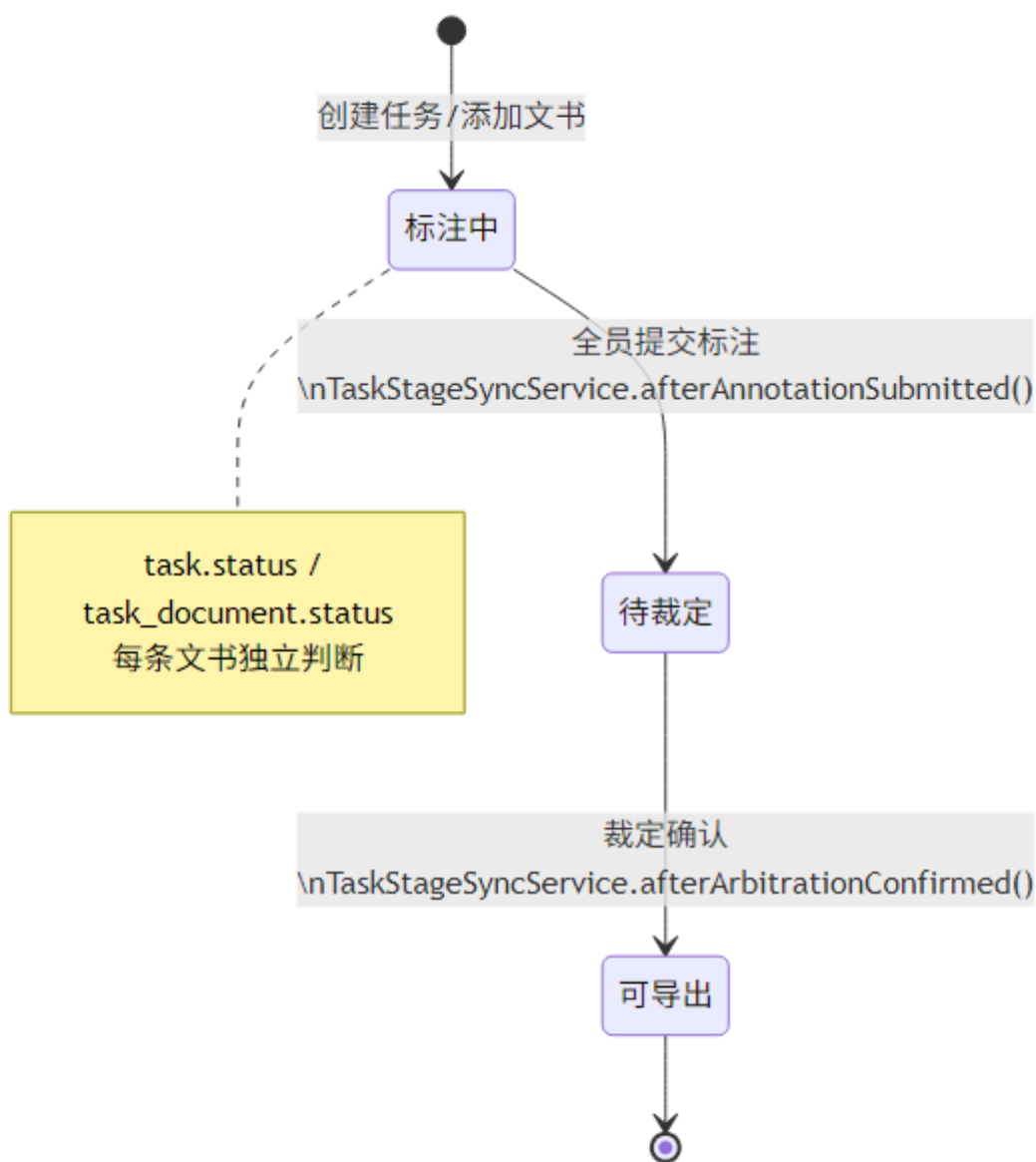
6.3 分层依赖方向

```

Controller → Service → Mapper → PO（数据库）
      ↓
      Entity/DTO（DomainConverter 转换）
    
```

所有 Service 通过 Spring 依赖注入 Mapper 接口，符合依赖倒转（面向 MyBatis 接口编程）。

七、任务阶段状态流转



任务阶段流转

Mermaid 源文件：P3-任务阶段流转.mmd

同步逻辑由 `TaskStageSyncService` 负责：

- `afterAnnotationSubmitted()`：检查每条文书是否全员提交
- `afterArbitrationConfirmed()`：将文书标记为可导出，并汇总任务状态